

Demand Forecasting and Product-Code Continuity

Business Understanding – P&G Case

Paula Banach – Student index 440186

Igor Kolodziej – Student index 440239

Mikita Silivestrau – Student index 392905

1 Introduction and Project Context

Demand forecasting is only useful when the data behind it is reliable. A forecast normally looks at past sales and uses them to estimate future demand. This sounds simple, but product data in large companies is not always stable. A product can receive a new code after reformulation, repackaging, regulation changes, or market restructuring. When this happens, the new code can lose the visible history of the old code.

The business problem in this project is product-code continuity. A product may still be the same or very similar from the customer point of view, but the system may treat it as a completely new product. For demand forecasting, this is a serious issue. The old code contains useful sales history, while the new code may look like it has no past. This creates a cold-start problem even when the product is not really new.

The proposed solution is a “Chains” system. A chain links an old product code to a new product code, so both codes can belong to one product family. Instead of forecasting only from the current code, a company can forecast from the full family history. This gives planners a clearer view of demand across code changes.

The project was implemented as a Flask adapter. The official test script called the local Flask server on port 5050. The adapter forwarded requests to a Django backend at:

`https://durczok.ovh/chains`

The API covered setup, event ingestion, validation, product-family recomputation, resolve, reverse resolve, and bulk resolve. The final test result was successful: 75 out of 75 tests passed, 0 failed, with a 100% score.

This report is an extended written version of the pitch presentation. It keeps the same main logic, but adds the business, data science, preparation, implementation, and testing details that could not fit into a five-minute presentation.

2 Business Idea and Data Science Value

2.1 The business problem

Demand forecasting supports many important decisions. It affects how much a factory should produce, how much inventory should be held, how goods should be sent to warehouses, and how sales plans should be built. If the forecast is too low, the company can face stockouts and lost sales. If it is too high, the company can produce too much and hold expensive inventory.

Historical sales data is one of the most important inputs for a forecast. A model can learn the normal sales level, seasonal patterns, growth or decline, and the effect of promotions. This only works well when the historical data belongs to the same business object. Product-code changes break this continuity.

The case study uses two code types:

Code type	Meaning
IPC	Internal Product Code, used inside the company.
GTIN	Global Trade Item Number, often used as a retail barcode.

Both code types can change for normal business reasons. A shampoo may receive a new internal code when its formula changes. A bottle may get a new GTIN after the package size changes. A product may also change because of new labeling rules or because two regional versions are merged into one standard version.

Without a chain, the new code can start with zero visible demand history. The old code still has history, but the forecasting system may not connect it to the new code. The system then sees a technical data change as a new product launch, even when the business situation is closer to product continuation.

2.2 Value for supply chains and consumer-goods companies

For a large consumer-goods company such as P&G, this problem can appear many times across countries, brands, package sizes, and product variants. The same type of product change may happen in Poland at one time and in Germany at another time. One country may use one code history, while another country may have a different chain.

Product-code continuity helps different parts of the business:

Area	Why continuity matters
Production planning	Factories need realistic demand estimates before planning materials, capacity, and shifts.
Inventory management	Missing history can lead to too much stock or stockouts after a product relaunch.
Distribution	Warehouses and stores need forecasts for the real product family, not only for the newest code.
Sales and finance	Forecasts affect revenue plans, targets, and budget decisions.

The Chains system gives the company a more stable business object: the product family. Individual codes still matter, because they show which exact code was active at a certain time. The family, however, preserves the wider product history.

For example, a product can move through three codes:

1001 -> 1002 -> 1003

If the current code is 1003, the family can still include demand from 1001 and 1002. A planner can then see the product as a continuing item, not as a new code without history.

2.3 Data science value

From a data science point of view, chains improve the quality of model input data. Forecasting models need clean time series. A time series for only the current code may be short and misleading. A time series for the product family can be longer and more stable.

The case study uses the concept of a generation. A generation is the lifespan of one product code. It has a code, a start date, and an end date. If a code is still active, the end date can be open-ended, represented in the case study as 9999-12-31.

Generation	Code	Start date	End date
1	1001	2021-01-01	2022-06-30
2	1002	2022-07-01	2024-03-31
3	1003	2024-04-01	9999-12-31

This time validity is important. A model dataset must not use a code before it existed. It must also know which code was active on each date. Generations make this possible.

Real product history is not always a straight chain. Products can split or merge. The case study therefore represents a product family as a Directed Acyclic Graph, or DAG. The direction goes from predecessor to successor. The graph should not contain loops, because a product cannot be both before and after itself in the same history path.

Pattern	Example	Meaning
Straight chain	A -> B -> C	One product continues through several codes.
Split	A -> B and A -> C	One old product becomes two successors.
Merge	A -> C and B -> C	Two old products become one successor.

This structure can support forecasting models in several ways. It can create longer family-level time series, reduce cold-start problems, and help models use predecessor history when a code changes. It can also support features such as “after split”, “after merge”, or “generation number”. These features may help the model understand that a demand pattern changed because of a product lifecycle event.

In a practical forecasting pipeline, the Chains output would usually be used before model training. Sales records would first be joined to product codes, countries, code types, and dates. The resolve endpoint could then add the product-family identifier to each record. After that, the data team could aggregate sales by product family and time period, for example by week or month. This would create a cleaner time series where the business product remains stable even when the technical product code changes.

This preparation step also helps avoid two common mistakes. The first mistake is losing old demand after a code change. The second mistake is joining history to a code outside its

valid date range. Generations reduce this risk because each code has a defined active period. A model can still use the exact active code as a feature, but the family gives the model a stronger historical base.

The system does not decide how to divide demand after every split or merge. That decision may still need business rules. For example, after a split, the historical demand of the predecessor may need to be allocated between two successor products. After a merge, the model may need to combine the histories of both predecessors. The Chains system provides the structure needed for these decisions, while the final modeling choice stays with the forecasting team.

The project did not train a forecasting model, so it does not report forecast accuracy results. In a later evaluation, suitable metrics would include MAE, RMSE, MAPE, and forecast bias. A fair test would compare forecasts using only current-code history against forecasts using family-level history.

3 Preparation and API Requirements

3.1 Understanding the assignment

The team first reviewed the pitch material, the case study, the implementation files, and the official-style test script. The most important files were:

File	Purpose
CASE_STUDY.md	Explained the business problem, domain concepts, validation rules, and API flow.
server.py	Contained the Flask adapter that exposes the local API.
test_api.py	Sent setup, event, recompute, resolve, and reverse-resolve requests.
mikita_final_test_output.txt	Stored the final evidence that all 75 tests passed.
PROJECT_WORK_SPLIT.md	Described the team responsibilities and handover points.

The team needed to understand both the business logic and the exact API behavior. The tests checked not only returned data, but also HTTP status codes. Valid events had to be accepted. Invalid events had to be rejected with HTTP 400. The adapter could not hide or rewrite backend responses without changing the meaning of the API.

3.2 Reference data and event types

The setup phase required a fixed set of countries: Poland, Germany, United States, Japan, and China. It also required two code types: IPC and GTIN. These values were seeded during

setup so later events could reference valid countries and code types.

Events are business occurrences in one country. Each event contains one or more transitions. The three transition types are:

Transition	Required fields	Effect
INTRO	introduction_code, date	Creates a new active generation.
DISCONT	discontinuation_code, date	Ends an active generation.
chain	introduction_code, discontinuation_code, date	Links predecessor and successor codes.

The test script created valid chains, splits, and merges. It also sent invalid cases to check the validation rules. The main invalid categories were:

- double introduction of an already active code;
- overlapping generation for the same code;
- double discontinuation or discontinuation of a code that is not active;
- chain with a missing or non-existing code;
- chain where the introduction code and discontinuation code are the same;
- missing date or missing required code field;
- invalid country code or invalid code type.

3.3 API flow

The project followed three main phases.

Phase	Action	Expected result
Setup	POST /api/setup/ resets event state and ensures reference data exists.	HTTP 200
Event ingestion	POST /api/events/ creates valid events and rejects invalid ones.	HTTP 201 or 400
Recompute and query	POST /api/product-families/recompute/, then resolve and reverse-resolve queries.	Correct families

This flow matches the real business process. First the system prepares reference data. Then it receives product lifecycle events. After that, it recomputes families. Finally, other systems can query which family a code belongs to, or which codes are active in a family on a date.

The work split followed the same flow. Igor focused on requirements, architecture, setup, and API foundation. Paula focused on event ingestion and validation behavior. Mikita focused on product-family recomputation, resolution endpoints, final testing, and slide assembly. This split made the project easier to organize because each part had a clear owner.

4 Implementation, Testing, and Conclusion

4.1 Flask adapter and backend forwarding

The implemented solution is technically simple on purpose. The Flask adapter exposes the API expected by the test script and forwards requests to the Django backend. The backend performs storage, validation, recomputation, and query logic. The adapter keeps the local interface stable.

The main configuration in `server.py` is:

```
DJANGO_URL = os.getenv("DJANGO_URL", "https://durczok.ovh/chains").rstrip("/")
REQUEST_TIMEOUT_SECONDS = int(os.getenv("REQUEST_TIMEOUT_SECONDS", "30"))

COUNTRIES = [
    ("PL", "Poland"),
    ("DE", "Germany"),
    ("US", "United States"),
    ("JP", "Japan"),
    ("CN", "China"),
]

CODE_TYPES = [
    ("IPC", "Internal Product Code"),
    ("GTIN", "Global Trade Item Number"),
]
```

The setup endpoint deletes old events and ensures that countries and code types exist. This makes test runs repeatable. If old events stayed in the backend, a new test run could be affected by old data.

One key design choice was to preserve backend responses. This matters because the tests expect specific status codes.

```
def _backend_response(response: requests.Response) -> Response:
    """
    Preserve backend response body, status code, and content type.
    """
    return Response(
        response.content,
        status=response.status_code,
        content_type=response.headers.get("Content-Type", "application/json"),
```

```
)
```

This helper prevents the adapter from changing a valid event response from 201 to another status code, or from hiding a validation error. It also keeps the backend response body available to the caller.

Forwarding is handled with small proxy helpers. For example:

```
def _proxy_post(path: str, data: dict[str, Any] | None = None) -> Response:
    try:
        response = requests.post(
            f"{DJANGO_URL}{path}",
            json=data if data is not None else {},
            timeout=REQUEST_TIMEOUT_SECONDS,
        )
    except requests.RequestException as exc:
        return _backend_error_response("POST", path, exc)

    return _backend_response(response)
```

This function sends JSON data to the backend, applies a timeout, and returns a clear 502 response if the backend cannot be reached. Similar helpers are used for GET and DELETE requests.

4.2 Events, recomputation, and queries

Event creation is a direct pass-through to the backend:

```
@app.route("/api/events/", methods=["POST"])
def create_event():
    return _proxy_post("/api/events/", _json_body())
```

The adapter does not duplicate the full validation logic. It forwards the event and keeps the backend result. This keeps the Flask layer small and reduces the risk of two systems applying validation differently.

After events are ingested, product families must be recomputed:

```
@app.route("/api/product-families/recompute/", methods=["POST"])
def recompute():
    return _proxy_post("/api/product-families/recompute/", _json_body())
```

Conceptually, recomputation turns stored events into generations, generation links, and product families. The pitch presentation described this as:

Events -> Generations -> Generation Links -> Product Families -> Resolve

The final query endpoints are:

Endpoint	Purpose
GET /api/resolve/	Finds the product family for a code, country, code type, and date.
GET /api/resolve/reverse/	Finds active codes in a product family on a given date.
POST /api/resolve/bulk/	Resolves many queries in one request.

These endpoints create the value for forecasting and reporting systems. A data pipeline can resolve a code to its stable family before building a time series. It can also reverse-resolve a family to find which codes were active on a given date.

4.3 Final testing

The final test run checked all required phases. Setup returned HTTP 200. The script created 100 test families across the supported countries. It checked 13 invalid-event scenarios, and all were rejected with HTTP 400. Recompute returned HTTP 200. Resolve and reverse-resolve queries returned the expected product-family behavior.

Test area	Result
Setup	POST /api/setup/ returned HTTP 200.
Event ingestion	100 families were created.
Invalid events	13 invalid cases returned HTTP 400.
Recompute	Product-family recomputation returned HTTP 200.
Queries	Resolve and reverse-resolve checks passed.
Final score	75 passed, 0 failed, 100%.

The final output was:

```
=== RESULTS ===  
Passed: 75  
Failed: 0  
Score: 75/75 (100%)
```

The test result is important because it checks the complete workflow, not only one endpoint. The setup phase shows that the system can start from a clean state, remove old event data, and prepare the required reference data before new events are sent. The invalid-event phase shows that the API protects product-family data from common mistakes, such as double introductions, missing fields, invalid countries, invalid code types, and chains that point to wrong codes. This matters because one incorrect event could connect the wrong histories and later create misleading forecasting input. It also checks that response status codes

stay meaningful, which is important for automated systems that must react differently to accepted and rejected events. The recompute phase shows that stored events can be turned into generations, links, and product families after ingestion. The resolve and reverse-resolve checks show that another system could ask practical questions, such as which family a code belongs to on a date, or which codes are active in a family. Together, these phases confirm that the adapter and backend can support the Chains idea as an API workflow. They also keep the result clear: the tests confirm API correctness and product-family resolution, not measured forecast accuracy.

This result confirms that the API matched the assignment tests. It does not show that forecast accuracy improved, because no model was trained in this project. It shows that the product-code continuity workflow works correctly according to the required API behavior.

4.4 Risks, limitations, and conclusion

The main limitation is backend dependence. The Flask adapter cannot work normally if the Django backend is unavailable. The adapter returns a clear backend error, but it cannot recompute or resolve families without the backend.

Another limitation is data quality. Product families are only correct if the incoming events are correct. If a user links the wrong predecessor and successor, the family history will be wrong. Event order and date quality also matter, especially for splits, merges, and reintroductions.

In a production operating model, chain events should be created by product or master-data owners, because they know when a code change represents a continuation rather than a real new product. Supply-chain planners should review important chains before they are used in forecasting, especially for splits and merges where demand allocation is sensitive. Each event should also be auditable, with a clear source, date, and responsible person, so later teams can understand why a family was built in a certain way.

The validation rules reduce many risks, but validation is still complex. A real production version would need audit logs, user-friendly validation messages, access control, and careful protection around setup or reset operations. A future data science phase would also need real sales data and a fair evaluation of forecast accuracy using metrics such as MAE, RMSE, MAPE, and forecast bias.

The project achieved its main goal. It turned the pitch idea into a working API workflow for product-code continuity. Product-code changes no longer have to break visible demand history. By linking old and new codes into product families, the system gives companies a cleaner base for demand forecasting, supply-chain planning, and business analysis.